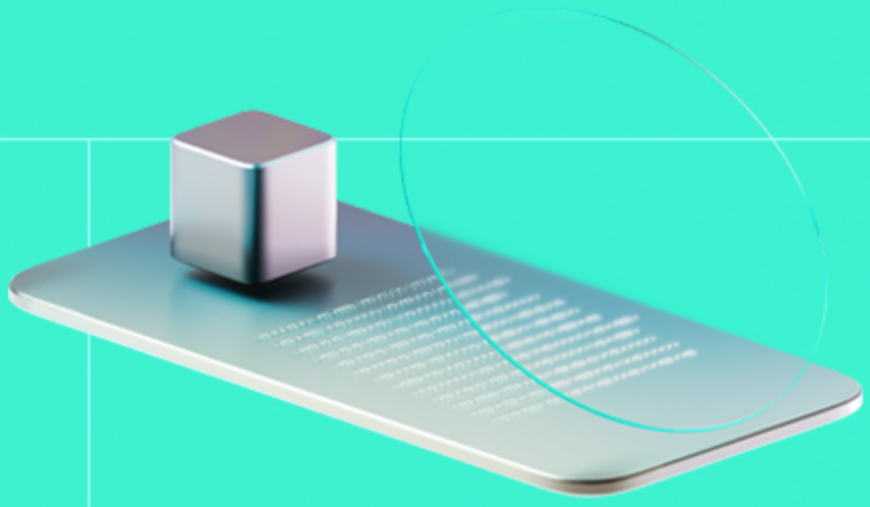
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: Panini America

Date: 8/1/2026

We express our gratitude to the Panini America team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Panini is an on-chain collectibles platform that allows authorised operators to mint NFTs, and users to lock NFTs for bridging, and unlock escrowed tokens via signature-verified requests while enforcing marketplace-whitelisted trades. It also manages royalty funds through a dedicated custody vault that supports controlled withdrawals and Uniswap-based token swaps.

Document

Name	Smart Contract Code Review and Security Analysis Report for Panini America
Audited By	Panagiotis Konstantinidis, Ataberk Yavuzer
Approved By	Ivan Bondar
Website	https://nft.paniniamerica.net
Changelog	06/10/2025 - Preliminary Report
	28/10/2025 - Remediation Report
	08/01/2026 - Final Report
Platform	EVM
Language	Solidity
Tags	Non-fungible Token (NFT), Signatures, ERC721
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/paniniamerica/SmartContracts
Initial commit	4b9b35d
Remediation commit	f070187
Final commit	68c618b
NFT Contract Proxy Address	https://etherscan.io/address/0x23ae7a05f598fc234ee9dbef04033080dea8ab19

Review Scope

NFT Contract Contract Implementation Address:	https://etherscan.io/address/0x290ae017831470 5d95e504ea94ce052fb550b613
Royalty Vault Contract Proxy Address:	https://etherscan.io/address/0x2033b17894bd32 af9297246827d43d9d67260af3
Royalty Vault Contract Implementation Address:	https://etherscan.io/address/0x71cf823dfeb46dc 24ed16d91b8b5fcc7387a2c4e

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

10	9	1	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	1
High	0
Medium	1
Low	5

Vulnerability	Severity	Status
F-2025-13249 - Transfer/Approval Bypass and DoS Due To Missing Access Control	Critical	Fixed
F-2025-13251 - Lack of Contract Address in Signed Payload Leads to Cross-Collection Replay Risk	Medium	Fixed
F-2025-13252 - Global Nonce Scope Causes Cross-User and Cross-Flow DoS During Signature Paths	Low	Accepted
F-2025-13253 - Missing onlyInitializing Guard on Internal Initializer Enables Re-Invocation Outside Init Phase	Low	Fixed
F-2025-13254 - The isApprovedForAll() Function Reverts on Read for Non-Whitelisted Operators	Low	Fixed
F-2025-13259 - Batch Mint Uses _mint Instead of _safeMint Risking Stuck NFTs on Contract Recipients	Low	Fixed
F-2025-13274 - Missing Deadline In UniswapV3 Swaps Allows Delayed Or MEV Exploited Execution	Low	Fixed
F-2025-13250 - Unused two-parameter _validateTransfer Function Creates Dead Code and Potential Confusion	Info	Fixed
F-2025-13256 - Empty try/catch block in _registerTokenType Function Hides Validator Failures	Info	Fixed
F-2025-13272 - NatSpec Inconsistencies and Documentation Issues	Info	Fixed

Documentation quality

- Functional requirements are clearly outlined within the documentation and NatSpec comments across all contracts.
- Technical descriptions and intended behaviour are included for core functionalities.
- Inline comments are consistently used throughout the codebase, improving readability and developer understanding.

Code quality

- Contract inheritance and modularity are well-structured, allowing separation between NFT logic, access control, validation, and royalty vault functionalities.
- The implementation uses sufficient input validation, role-based access modifiers, and event logging for traceability.
- Several template code patterns from OpenZeppelin (Ownable, AccessControl, Pausable, ReentrancyGuard) are implemented.

Test coverage

Code coverage of the project is **39.4%** (branch coverage).

- Negative cases are partially covered.
- A comprehensive set of tests validates deployment, minting, role-based access, pausing, burning, approvals, and signature-based operations.
- The tests execute correctly after uncommenting specific sections as instructed by the development team.

Table of Contents

System Overview

The **Panini Protocol** is an on-chain collectibles and royalty management system designed to enable secure minting, transfer, locking, and unlocking of NFTs through cryptographic validation and managed fund custody. It allows authorized operators to mint NFTs, and users to bridge them across networks by locking or unlocking assets via signatures, enforce marketplace whitelisting for compliant trading, and manage royalty proceeds using a secure, role-based vault.

PaniniNFTs — an upgradeable ERC-721 collection with royalties and strict, role-based controls. It supports operator-gated minting (single or batch), optional burning with burned token ID reuse protection, and signature-based mint/unlock flows that use nonces and expirations for replay resistance. The lock flow escrows tokens to the contract for bridging, while transfers and approvals are filtered through the Panini lock and marketplace whitelisting (SmartValidator) as well as the external transfer-policy hook (CreatorTokenValidator). Managers can also pause/unpause operations and update token URIs when necessary.

SmartValidator — is an access-controlled validation module enforcing marketplace whitelisting and Panini lock logic, ensuring that only approved operators or marketplaces can execute transfers and approvals when the lock is active.

CreatorTokenValidator — is an abstract module for enforcing customizable transfer validation policies, allowing integration with external transfer validators and ensuring compatibility with the LimitBreak creator token standard.

PaniniRoyaltyVault — is a secure upgradeable vault contract that manages both ETH and ERC20 funds, allowing authorized roles to withdraw or swap tokens through Uniswap, while maintaining pausing, access control, and non-reentrancy safeguards.

Privileged roles

Privileged roles of the PaniniNFTs contract:

- The `DEFAULT_ADMIN_ROLE` role can grant and revoke all AccessControl roles used by the NFT system, including `PANINI_NFT_MANAGER`, `PANINI_NFT_OPERATOR`, and `WHITELISTED_MARKETPLACE`, thereby controlling the contract's role-based access system.
- The `PANINI_NFT_MANAGER` role can pause and unpause transfers, force-update token metadata via updateTokenURI, and freeze or unfreeze specific user accounts for this NFT collection via the configured external transfer validator.
- The `PANINI_NFT_OPERATOR` role can mint and batch mint NFTs to users (subject to the global minting flag), and also acts as the signing authority for the signature-based bridging flows (`batchMintOrUnlock` and `batchLockNFT`), since signatures are accepted only if recovered signer holds `PANINI_NFT_OPERATOR`.
- The `owner` can configure royalties, toggle minting and burning permissions globally, and set or change the external transfer validator that enforces the collection's transfer-policy rules (including account freeze/unfreeze behavior).

Privileged roles of the PaniniRoyaltyVault contract:

- The `DEFAULT_ADMIN_ROLE` role can grant and revoke all vault roles (including `VAULT_MANAGER`, `VAULT_PAUSER`, and `WHITELISTED_RECEIVER`) and can update the Uniswap router address via `updateUniswapRouter()` function.
- The `VAULT_MANAGER` role can withdraw ETH and ERC20 tokens to whitelisted receivers and can execute swaps (ETH to ERC20 and ERC20 to ERC20) through the configured Uniswap V3 router.
- The `VAULT_PAUSER` role can pause and unpause vault operations, restricting withdrawals and swaps during maintenance or security incidents.
- The `owner` can transfer or renounce ownership; additionally, the owner is granted `DEFAULT_ADMIN_ROLE` during initialization, so the owner is the initial authority for vault role administration unless this is later reconfigured.

Privileged roles of the SmartValidator contract:

- The `WHITELISTED_MARKETPLACE` role allows approved marketplace or operator addresses to perform approvals and initiate transfers while the Panini lock mechanism is active, bypassing the default restriction that otherwise permits only token owners to perform these actions.
- The `DEFAULT_ADMIN_ROLE` role can enable or disable the Panini lock mechanism globally via `setPaniniLock()` function, directly affecting whether approvals and transfers are gated by WHITELISTED_MARKETPLACE restrictions.

Privileged roles of the CreatorTokenValidator contract:

- The `owner` can set or change the transfer validator contract via `setTransferValidator(address)`, thereby defining (or replacing) the collection's external transfer-policy enforcement logic that is invoked on transfers and is also used to apply administrative actions such as freezing and unfreezing accounts for the collection.

Potential Risks

- **Centralized Control of Minting, Trading Permissions, and Funds:** Privileged roles (`PANINI_NFT_OPERATOR`, `PANINI_NFT_MANAGER`, `DEFAULT_ADMIN_ROLE`, `VAULT_MANAGER`) can mint/batch-mint NFTs, pause/unpause transfers via the `paniniLock` and pauable modules, update metadata, freeze or unfreeze specific accounts for the collection, and withdraw or swap assets from `PaniniRoyaltyVault`. Compromise or misuse of these keys can censor trading or drain funds.
- **Misconfiguration or Compromise of Signature Authority:** The `batchMintOrUnlock()` and `batchLockNFT()` functions rely on an off-chain signer (address holding `PANINI_NFT_OPERATOR`). If that key is leaked or rotated improperly, attackers could mint/unlock/lock arbitrary tokens bypassing the current checks.
- **Reliance on External Transfer Validator:** The `setTransferValidator()` function of the `CreatorTokenValidator` contract lets the `owner` point the collection to an external validator. A malicious or faulty validator could block legitimate transfers or silently allow restricted flows, altering collection policy at any time.
- **Marketplace Whitelisting & Panini Lock Risks:** Trading requires either ownership or a whitelisted marketplace (`WHITELISTED_MARKETPLACE`) while `paniniLock` is true. Incorrect role assignments or lock toggling can unintentionally halt secondary-market activity or enable unvetted operators.
- **Custodial Vault & Router Dependency:** The `PaniniRoyaltyVault` contract holds ETH and ERC20 assets and performs swaps through an external router (`IUniswapV3Router3`). Router bugs, pool manipulation, or slippage can lead to losses, and a compromised `VAULT_MANAGER` can withdraw tokens arbitrarily.
- **Absence of On-chain Time-locks for Critical Changes:** Immediate execution is possible for actions like granting/revoking roles, toggling `paniniLock`, freezing/unfreezing accounts, changing royalties, and updating the transfer validator/router. Without a delay, users have no window to react to harmful changes.
- **Flexibility and Risk in Contract Upgrades:** The project's contracts are upgradeable, allowing authorized administrators to update contract logic to remediate vulnerabilities or implement approved business rule changes. While this provides necessary flexibility for issue resolution and future enhancements, it introduces inherent governance risk if upgrade authority is ever misused or compromised (e.g., key compromise, misconfiguration, or execution outside the intended process), potentially enabling unsafe or unauthorized logic changes. Panini America noted that upgrades are restricted to remediation or formally approved business updates and are subject to strict controls, including an independent re-audit by Hacken, security review and validation of the upgraded logic, and formal approval prior to executing any production upgrade. This governance process materially reduces upgrade-related risk, but the upgrade mechanism remains a privileged capability whose security depends on continued adherence to these controls and secure management of the upgrade authority.
- **Absence of Upgrade Window Constraints:** The contract suite permits upgrades to be executed immediately by authorized administrators, without an on-chain enforced timelock or mandatory waiting period. While this can be beneficial for rapid remediation of critical issues, it also increases the inherent governance risk that malicious or flawed logic could be deployed quickly if upgrade authority is misused or compromised. Panini America

noted that upgrades are restricted to remediation of identified vulnerabilities or formally approved business rule changes and are subject to strict governance controls, including an independent re-audit by Hacken covering the proposed changes, security review and validation of the upgraded logic, and formal approval prior to executing any production upgrade. This off-chain governance framework materially reduces the practical likelihood of unsafe rapid upgrades, but because the upgrade mechanism lacks protocol-level delay constraints, system integrity continues to depend on secure management of privileged upgrade keys and consistent adherence to Panini America's defined approval and audit process.

- **Insufficient Multi-signature Controls for Critical Functions:** Sensitive operations are restricted to the owner account. Panini America clarified that ownership is held by a Safe (multi-signature) wallet, meaning critical actions require confirmation by all or a majority of the Safe's owners (per the configured threshold). This materially reduces single-key concentration risk and improves operational security compared to an EOA owner model. However, overall safety still depends on maintaining the Safe's signer set and threshold securely (e.g., preventing signer compromise, ensuring appropriate quorum settings, and applying strong operational procedures for key management and approvals).

Findings

Vulnerability Details

F-2025-13249 - Transfer/Approval Bypass and DoS Due To Missing Access Control - Critical

Description:

The `setPaniniLock()` function in the `SmartValidator` contract lacks proper access control, allowing any external address to enable or disable the Panini Lock mechanism. This vulnerability has severe security implications as the Panini Lock is a core security feature that controls NFT transfer and approval restrictions throughout the protocol.

When the Panini Lock is enabled, it enforces that only token owners or whitelisted marketplaces can perform transfers and approvals. However, due to the missing access control, malicious actors can:

1. **Bypass all transfer restrictions** by disabling the Panini Lock, allowing unauthorized transfers of NFTs to any address, completely undermining the protocol's security model.
2. **Cause Denial of Service (DoS)** by enabling the Panini Lock when it should be disabled, preventing legitimate users from transferring their NFTs even to themselves or approved marketplaces.

This vulnerability compromises the entire security architecture of the NFT protocol, as the Panini Lock mechanism is fundamental to preventing unauthorized transfers and maintaining marketplace control.

```
function _validateTransfer(address caller, address from) internal view {
    if (paniniLock) {
        if (!hasRole(WHITELISTED_MARKETPLACE, caller) && caller != from) {
            revert InvalidOperator(
                "Caller Is Not Owner Whitelisted Marketplace "
            );
        }
    }
}

/// @notice Validates if an approval action is allowed
function _validateApproval(address _operator) internal view {
    if (paniniLock) {
        if (!hasRole(WHITELISTED_MARKETPLACE, _operator)) {
```

```
            revert InvalidOperator(
                "Operator/Marketplace is not whitelisted"
            );
        }
    }
}

/// @notice Enable or disable Panini Lock
function setPaniniLock(bool _status) public  {
    paniniLock = _status;
}
```

Assets:

- contracts/PaniniNfts/smartlocks/SmartValidator.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Critical

Recommendations

Remediation: Implement proper access control by restricting the `setPaniniLock()` function to authorized roles only. The function should be protected with an appropriate access control modifier:

```
/// @notice Enable or disable Panini Lock
function setPaniniLock(bool _status) public onlyRole(DEFAULT_ADMIN_ROLE) {
    paniniLock = _status;
}
```

Alternatively, if a more granular approach is needed, create a specific role for managing the Panini Lock:

```
bytes32 public constant PANINI_LOCK_MANAGER = keccak256("PANINI_LOCK_MANAGER"
);
```

```
/// @notice Enable or disable Panini Lock
function setPaniniLock(bool _status) public onlyRole(PANINI_LOCK_MANAGER) {
    paniniLock = _status;
}
```

Resolution: The finding was resolved in commit `f070187` after the `onlyRole(DEFAULT_ADMIN_ROLE)` modifier was included.

Evidences

PoC

Reproduce:

Test case:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {Test, console} from "forge-std/Test.sol";
import {SmartValidator} from "../src/PaniniNfts/smartlocks/SmartValidator.sol
";

contract TestSmartValidator is SmartValidator {
    constructor(address admin) {
        paniniLock = true;
        _grantRole(0x00, admin); // DEFAULT_ADMIN_ROLE is 0x00
    }

    // Expose internal functions for testing
    function validateTransferPublic(address caller, address from, address to)
public view {
        _validateTransfer(caller, from, to);
    }

    // Simulate token transfer scenarios
    function simulateTransfer(address caller, address from, address to) publi
c view returns (bool success) {
        try this.validateTransferPublic(caller, from, to) {
            return true;
        } catch {
            return false;
        }
    }
}

contract SmartValidatorVulnerabilityTest is Test {
    TestSmartValidator public validator;
```

```
    address public admin = makeAddr("admin");
    address public attacker = makeAddr("attacker");
    address public user1 = makeAddr("user1");
    address public user2 = makeAddr("user2");
    address public marketplace = makeAddr("marketplace");

    function setUp() public {
        validator = new TestSmartValidator(admin);

        // Admin adds a whitelisted marketplace
        vm.startPrank(admin);
        validator.grantRole(validator.WHITELISTED_MARKETPLACE(), marketplace)
;
        vm.stopPrank();
    }

    function test_CriticalVulnerability_AnyoneCanDisablePaniniLock() public {
        // Initially, Panini Lock is enabled (default state)
        assertTrue(validator.paniniLock());

        // Non-whitelisted transfers should be blocked
        assertFalse(validator.simulateTransfer(attacker, user1, user2));

        // VULNERABILITY: Any random address can disable Panini Lock
        vm.prank(attacker);
        validator.setPaniniLock(false);

        // Verify the lock is now disabled
        assertFalse(validator.paniniLock());

        // Now the same non-whitelisted transfer is allowed, bypassing securi
ty
        assertTrue(validator.simulateTransfer(attacker, user1, user2));

        console.log("CRITICAL: Attacker successfully disabled Panini Lock!");
        console.log("All transfer restrictions have been bypassed!");
    }

    function test_CriticalVulnerability_AttackerCanEnableLockToDoS() public {
        // Admin disables lock for some reason
        vm.startPrank(admin);
        validator.setPaniniLock(false);
        vm.stopPrank();

        // All transfers are now allowed
        assertTrue(validator.simulateTransfer(attacker, user1, user2));

        // VULNERABILITY: Attacker can re-enable lock to cause DoS
        vm.prank(attacker);
```

```
        validator.setPaniniLock(true);

        // Now non-whitelisted transfers are blocked again
        assertFalse(validator.simulateTransfer(attacker, user1, user2));

        console.log("CRITICAL: Attacker enabled Panini Lock, causing DoS!");
        console.log("Protocol functionality has been disrupted!");
    }
}
```

Results:

Output:

```
Ran 2 tests for test/SmartValidatorVulnerability.t.sol:SmartValidatorVulnerab
ilityTest
[PASS] test_CriticalVulnerability_AnyoneCanDisablePaniniLock() (gas: 34306)
[PASS] test_CriticalVulnerability_AttackerCanEnableLockToDoS() (gas: 38165)
```

F-2025-13251 - Lack of Contract Address in Signed Payload Leads to Cross-Collection Replay Risk - Medium

Description:

The `PaniniNFTs` contract's signature verification logic in `_verifySignature()` internal function builds the signed message without including the current collection's own contract address (`address(this)`) in the signature payload. The current encoded message is:

```
bytes memory message = abi.encode(
            block.chainid,
            _msgSender(),
            tokenIds,
            tokenURIs,
            requestNonce,
            expiredAt
        );

require(_verifySignature(message, signature), "Invalid signature");

...

    function _verifySignature(
        bytes memory message,
        bytes memory signature
    ) internal view returns (bool) {
        bytes32 messageHash = keccak256(message);
        bytes32 digest = MessageHashUtils.toEthSignedMessageHash(messageHash)
;
        address signer = ECDSA.recover(digest, signature);
        return hasRole(PANINI_NFT_OPERATOR, signer);
    }
```

The `_verifySignature()` ECDSA-recovers an address with `toEthSignedMessageHash` and checks the `PANINI_NFT_OPERATOR` role. Because `address(this)` is omitted, the recovered signature is not bound to a specific contract instance. This design means signatures are only bound to the chain ID, sender, and parameters, but not to the specific NFT collection contract.

If the same operator key is used across multiple collections on the same chain, (e.g., new deployments/clones as the project evolves, which is common with upgradeable architectures) then a valid signature generated for Contract A can be replayed against Contract B (as long as nonce conditions permit). This enables unauthorized

minting, unlocking, or locking actions on other collections that share operator keys.

As a result:

- Signatures can be replayed across different NFT contracts on the same chain.
- An attacker could mint or unlock NFTs in another collection without authorization if operator keys are reused.
- Trust boundaries between collections are weakened, undermining security assumptions of signature-based controls.

Assets:

- contracts/PaniniNfts/PaniniNFTs.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Strengthen the signature verification by including `address(this)`, `block.chainid`, and add a purpose discriminator (versioned string) so signatures can't be replayed across contracts or re-used for a different action after refactors. This modification should be applied to both `batchMintOrUnlock()` and `batchLockNFT()` functions that include a signature process.

Example Fix:

```
//batchMintOrUnlock() function
bytes memory message = abi.encode(
    address(this),                // contract domain
    block.chainid,                // chain domain
    "PANINI_MINT_UNLOCK_V1",      // action discriminator
    _msgSender(),
    tokenIds,
    tokenURIs,
```

```
    requestNonce,
    expiredAt
);
```

This ensures:

- Cross-contract replay is blocked (bound to `address(this)` and `chainid`).
- Cross-function replay is blocked (different action tags like `"PANINI_LOCK_V1"` vs `"PANINI_MINT_UNLOCK_V1"`).
- Future-proofing via versioned tags prevents older signatures from being valid after flow changes.

Resolution: The finding was fixed in commit hash `f070187`. The suggested fix was implemented for extra prevention.

F-2025-13252 - Global Nonce Scope Causes Cross-User and Cross-Flow DoS During Signature Paths - Low

Description: The `PaniniNFTs` contract tracks replay with a single global map:

```
mapping(uint256 => bool) public usedNonces;
...
require(!usedNonces[requestNonce], "Nonce already used");

bytes memory message = abi.encode(
      block.chainid,
      _msgSender(),
      tokenIds,
      requestNonce,
      expiredAt
  );
  require(_verifySignature(message, signature), "Invalid signature");
```

The signed payloads already bind intent to the caller (`_msgSender()`), so a signature for User A can't be replayed by User B cryptographically. However, the on-chain consumption check is keyed only by the numeric `requestNonce`. That makes the nonce namespace global across all users and flows, which creates accidental denial-of-service and coordination failures. Specifically:

1. Cross-user collision (coordination risk): If the operator unintentionally (by mistake, concurrency, or multi-service tooling) issues two valid signatures that both use `requestNonce = 51`, one for User A and one for User B, whichever user submits first flips `usedNonces[51] = true`. The second, perfectly valid authorization reverts with `Nonce already used.` No attacker is required, since this is an operational foot-gun caused by global scoping.
2. Cross-purpose collision (same user, different actions): If the operator reuse the same numeric nonce for distinct signed flows (e.g., `batchMintOrUnlock()` vs `batchLockNFT()`), the first submission burns the shared number and the other flow reverts, even though the messages differ. Adding an action discriminator to the signed data improves clarity but does not fix this unless the consumption key also changes.

This is not a signature spoofing bug, but it's a fragility in replay protection that can cause valid, authorized requests to fail unpredictably under normal operations.

Assets:

- contracts/PaniniNfts/PaniniNFTs.sol [https://github.com/paniniamerica/SmartContracts]

Status: Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Adopt a replay guard that scopes consumption to the user address and the exact message, removing cross-user and cross-flow collisions.

Example fix:

- Scope by user:

```
mapping(address => mapping(uint256 => bool)) public usedNonce;

// when consuming
require(!usedNonce[_msgSender()][requestNonce], "Nonce used");
usedNonce[_msgSender()][requestNonce] = true;
```

This eliminates cross-user clashes. (If you have multiple signed flows per user, use distinct nonce spaces per flow or include a flow key in the mapping.)

Or:

- Mark the signed digest:

```
// Build message with strong domain separation + purpose tag
bytes memory message = abi.encode(
    address(this),
    block.chainid,
    "PANINI_MINT_UNLOCK_V1",  // or "PANINI_LOCK_V1"
    _msgSender(),
    tokenIds,
```

```
    tokenURIs,                // omit for lock flow
    requestNonce,
    expiredAt
);
bytes32 digest = keccak256(message);

mapping(bytes32 => bool) public usedDigest;
require(!usedDigest[digest], "Digest used");
usedDigest[digest] = true;
```

Digest-based replay protection makes each unique authorization one-time, automatically preventing cross-user and cross-purpose collisions. Combined with `address(this)` and `chainid` (and your action discriminator), it also reinforces domain separation and intent binding.

Resolution: **Accepted:** The project team acknowledged the risk, no action taken.

F-2025-13253 - Missing onlyInitializing Guard on Internal Initializer Enables Re-Invocation Outside Init Phase - Low

Description:

During `CreatorTokenValidator` contract, the internal initializer `__CreatorTokenValidator_init()` lacks the `onlyInitializing` modifier that OpenZeppelin's upgradeable pattern relies on to restrict helper initializers to the initializer call sequence. The current function it is defined as:

```
function __CreatorTokenValidator_init() internal {
    _emitDefaultTransferValidator();
    _registerTokenType(DEFAULT_TRANSFER_VALIDATOR);
}
```

Because it is not guarded by the `onlyInitializing` modifier, any inheriting contract could (now or in a future refactor) call this function again after deployment. While not an immediate external exploit, the function is emitting `TransferValidatorUpdated(address(0), DEFAULT_TRANSFER_VALIDATOR)` and making an external call to the validator via `_registerTokenType()`, so accidental re-entry later in the proxy lifecycle can:

1. produce misleading events,
2. re-register token type unexpectedly
3. and create brittle init-order assumptions for future upgrades.

This deviates from OZ's standard pattern:

```
function __X_init() internal onlyInitializing { ... }
```

which guarantees the helper can only run during `initializer` execution.

Assets:

- contracts/PaniniNfts/limitbreak/CreatorTokenValidator.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Restrict the `__CreatorTokenValidator_init()` to the initializer execution path and make it safe on repeated invocation. Add the `onlyInitializing` modifier and make it safe to call more than once without side effects. This keeps behavior aligned with OpenZeppelin expectations and prevents accidental re-execution after deployment.

Example Fix:

```
function __CreatorTokenValidator_init() internal onlyInitializing {
    _emitDefaultTransferValidator();
    _registerTokenType(DEFAULT_TRANSFER_VALIDATOR);
}
```

Resolution: The finding was resolved in commit hash `f070187a1a8eb96f28f8e9c3b4b7825f233b6a1e`. The `onlyInitializing` modifier was attached to the vulnerable function.

F-2025-13254 - The isApprovedForAll() Function Reverts on Read for Non-Whitelisted Operators - Low

Description:

The `PaniniNFTs` contract overrides `isApprovedForAll` (from `ERC721Upgradeable`/`IERC721`) and, before returning the approval status, calls the `_validateApproval(operator)` function:

```
// PaniniNFTs contract
function isApprovedForAll(
       address owner,
       address operator
   ) public view override(ERC721Upgradeable, IERC721) returns (bool) {
       _validateApproval(operator);
       return super.isApprovedForAll(owner, operator);
   }
```

When `paniniLock` is enabled, `_validateApproval()` reverts for operators that are not in `WHITELISTED_MARKETPLACE`:

```
//SmartValidator contract
function _validateApproval(address _operator) internal view {
    if (paniniLock) {
        if (!hasRole(WHITELISTED_MARKETPLACE, _operator)) {
            revert InvalidOperator(
                "Operator/Marketplace is not whitelisted"
            );
        }
    }
```

Because `isApprovedForAll` is a view query that wallets, marketplaces or indexers may call routinely, this turns a harmless read into a reverting call for non-whitelisted operators, breaking UI flows and off-chain indexing. This means that policy checks should be enforced on state-changing functions (`approve`, `setApprovalForAll`), and not on passive reads.

As a result wallets, marketplaces or indexers that query `isApprovedForAll()` for arbitrary operators will see unexpected reverts, causing broken user experience and integration issues.

Assets:

- contracts/PaniniNfts/PaniniNFTs.sol [https://github.com/paniniamerica/SmartContracts]
- contracts/PaniniNfts/smartlocks/SmartValidator.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Remove the enforcement of the `_validateApproval()` functionality during the read path and return `false` for non-whitelisted operators.

Example Fix:

```
function isApprovedForAll(address owner, address operator)
    public
    view
    override(ERC721Upgradeable, IERC721)
    returns (bool)
{
    // Non-reverting read: treat non-whitelisted operators as not approved
    if (paniniLock && !hasRole(WHITELISTED_MARKETPLACE, operator)) {
        return false;
    }
    return super.isApprovedForAll(owner, operator);
}
```

Resolution: The finding was fixed in commit hash `f070187` after the Panini America team implemented the suggested fix in the code.

F-2025-13259 - Batch Mint Uses _mint Instead of _safeMint Risking Stuck NFTs on Contract Recipients - Low

Description:

In `PaniniNFTs` contract, during the `batchMint()` function, each token is created with the raw `_mint(to, tokenId)` call. The raw `_mint` does not verify that the recipient (`to`) can handle ERC-721 tokens. If `to` is a smart contract that hasn't implemented the ERC-721 receiver interface (`onERC721Received`), the mint will still succeed, but the recipient contract won't recognize or be able to manage the NFT. In practice, this can stuck tokens inside incompatible contracts and break user flows (the token exists, but the recipient can't use or forward it).

Safe minting via `_safeMint()` function adds a crucial protection: after minting, it calls `onERC721Received` on the recipient if `to` is a contract. If the recipient doesn't implement the interface or returns the wrong selector, the transaction reverts, preventing NFTs from being minted into contracts that can't accept them.

```
function batchMint(
    address to,
    uint256[] memory tokenIds,
    string[] memory uris
) external onlyRole(PANINI_NFT_OPERATOR) {
    ...
    for (uint256 i = 0; i < tokenIds.length; i++) {
        require(!_exists(tokenIds[i]), "Token ID already exists");
        require(!burnedTokenIds[tokenIds[i]], "Token ID was burned and cannot
be reused");

        _mint(to, tokenIds[i]);                 // ← no onERC721Received che
ck
        _setTokenURI(tokenIds[i], uris[i]);
    }
}
```

As a result:

- NFTs can end up minted to contracts that cannot receive ERC-721s, leaving them effectively stuck.
- Users and integrators may see mints succeed on-chain but then be unable to interact with the tokens, causing confusing UX and support issues.

Assets:

- contracts/PaniniNfts/PaniniNFTs.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Use `_safeMint()` inside the loop so contract recipients are validated via `onERC721Received` and incompatible recipients cause a revert.

Additionally, Reentrancy is possible with `onERC721Received` because this **hook** is called during a safe mint, allowing the receiving contract to execute arbitrary code (including re-entering the ERC721 contract) before the transfer completes.. Therefore, it is also important to implement `nonReentrant` modifier for the `batchMint` function as `onERC721Received` function leads potential reentrancy attacks.

Example fix:

```
for (uint256 i = 0; i < tokenIds.length; i++) {
    require(!_exists(tokenIds[i]), "Token ID already exists");
    require(!burnedTokenIds[tokenIds[i]], "Token ID was burned and cannot be
reused");

    _safeMint(to, tokenIds[i]);                // ← safe: validates contract
recipients
    _setTokenURI(tokenIds[i], uris[i]);
}
```

Resolution: The internal `_mint` function was replaced with `_safeMint()` in commit hash `f070187`. Therefore, the finding is fixed.

F-2025-13274 - Missing Deadline In UniswapV3 Swaps Allows Delayed Or MEV Exploited Execution - Low

Description:

The `PaniniRoyaltyVault` contract implements token swaps via UniswapV3 in both `swapEthForToken()` and `swapTokenForToken()` functions. These functions construct the `IUniswapV3Router3.ExactInputSingleParams` struct and call `uniswapRouter.exactInputSingle` without supplying a `deadline` parameter. In addition, there is no local guard such as a `require(block.timestamp <= deadline)`.

```
function swapEthForToken(
        ...
    )
        ...
        IUniswapV3Router3.ExactInputSingleParams
            memory params = IUniswapV3Router3.ExactInputSingleParams({
                tokenIn: uniswapRouter.WETH9(),
                tokenOut: outToken,
                fee: feeTier,
                recipient: recipient,
                amountIn: amountIn,
                amountOutMinimum: amountOutMin,
                sqrtPriceLimitX96: sqrtPriceLimitX96 // 0 as default
            });
        // send ETH along with the swap
        amountOut = uniswapRouter.exactInputSingle{value: amountIn}(params);

        ...
    }


    function swapTokenForToken(
        ...
    )
        ...
        IUniswapV3Router3.ExactInputSingleParams
            memory params = IUniswapV3Router3.ExactInputSingleParams({
                tokenIn: inToken,
                tokenOut: outToken,
                fee: feeTier,
                recipient: recipient,
                amountIn: amountIn,
                amountOutMinimum: amountOutMin,
                sqrtPriceLimitX96: sqrtPriceLimitX96 // 0 as default
```

```
            });

        // ERC20 swap, no ETH needed
        amountOut = uniswapRouter.exactInputSingle{value: 0}(params);
        ...
    }
```

In Uniswap's reference router, the `deadline` field ensures that swaps must settle within a specified number of seconds after submission, preventing delayed or stale execution. In the current design, however, a submitted swap transaction remains valid indefinitely until mined.

This creates two distinct risks:

1. **MEV/Validator exploitation:** Validators or searchers can intentionally delay the inclusion of the swap and only execute it once the market has moved in their favor, provided the execution still satisfies the `amountOutMinimum` value. This can result in value leakage from the vault to MEV actors.
2. **Operational fragility:** Treasury operators may expect swaps to execute immediately, but due to the missing deadline, they can settle much later under different price conditions, complicating accounting and forecasting.

While `amountOutMinimum` does protect against unlimited slippage, it does not prevent adversarial timing, which undermines the user's intent to trade at "current" prices.

As a result:

- Vault swaps may execute at unfavourable times, leaking value to MEV or miners while still passing the minimum output check.
- Stale transactions can execute long after submission, breaking treasury assumptions and operational consistency.

Assets:

- contracts/PaniniRoyaltyVault/PaniniRoyaltyVault.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Adopt Uniswap's official V3 router interface that includes a `deadline`, and always pass a short expiration time (e.g., set `deadline` to `block.timestamp + 120` for a 2-minute window). This ensures the swap executes promptly or reverts if delayed, reducing MEV/timing risk.

Example Fix:

```
IUniswapV3Router.ExactInputSingleParams memory params = IUniswapV3Router.Exac
tInputSingleParams({
    tokenIn: inToken,
    tokenOut: outToken,
    fee: feeTier,
    recipient: recipient,
    deadline: block.timestamp + 120,   // 2-minute Time-To-Live (TTL)
    amountIn: amountIn,
    amountOutMinimum: amountOutMin,
    sqrtPriceLimitX96: sqrtPriceLimitX96
});
```

Resolution: This finding was resolved in commit hash `f070187`. The suggested `deadline` parameter is utilized in the swap function.

F-2025-13250 - Unused two-parameter _validateTransfer Function Creates Dead Code and Potential Confusion - Info

Description:

The `SmartValidator` contract declares two internal overloads of the `_validateTransfer()` function:

```
function _validateTransfer(address caller, address from, address to) internal view { /* active */ }
function _validateTransfer(address caller, address from)       internal view { /* duplicate */ }
```

Across the current codebase, only the three-parameter version is used (e.g., `PaniniNFTs._update` calls `_validateTransfer(auth, _ownerOf(tokenId), to)`). The two-parameter overload is never referenced. Keeping a dead, near-identical API in a security-critical validator adds maintenance overhead and ambiguity. Future updates may accidentally wire the wrong overload, causing incomplete policy checks and aadditionally marginally bloats bytecode.

As a result, future changes can unintentionally weaken or bypass the validator, leading to incorrect validator usage and subtle authorization gaps.

Assets:

- contracts/PaniniNfts/smartlocks/SmartValidator.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Remove the unused two-parameter overload, or explicitly forward it to the three-parameter path to keep a single source of truth. This reduces confusion, keeps validation logic centralized, and prevents inadvertent reliance on a stale or weaker code path.

Resolution: The finding is resolved in commit hash `f070187` after the unused two-parameter function was removed from the code.

F-2025-13256 - Empty try/catch block in _registerTokenType Function Hides Validator Failures - Info

Description:

In `CreatorTokenValidator` contract during the `_registerTokenType()` internal function, the external call to the validator's `setTokenTypeOfCollection()` is wrapped in an empty `try/catch` block. If the validator address is wrong, the interface is incompatible, or the call reverts at runtime, the failure is swallowed, since there's no revert, no event, and no diagnostic signal. This makes validator setup issues hard to detect in tests and production, and the collection can operate without the intended transfer-enforcement, while everyone believes registration succeeded.

```
function _registerTokenType(address validator) internal {
    if (validator != address(0)) {
        uint256 validatorCodeSize;
        assembly { validatorCodeSize := extcodesize(validator) }
        if (validatorCodeSize > 0) {
            try ITransferValidatorSetTokenType(validator)
                .setTokenTypeOfCollection(address(this), _tokenType())
            {} catch {}
        }
    }
}
```

As a result, misconfigurations can slip by and intended policies go unenforced, while operations or monitoring get no signal to investigate, delaying debugging and increasing its cost.

Assets:

- contracts/PaniniNfts/limitbreak/CreatorTokenValidator.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Surface failures so misconfigurations are visible and emit an event in `catch` block. Optionally add a strict mode for initialization where failures revert.

```
// Add an event
event TokenTypeRegistrationFailed(address indexed validator, address indexed
collection, bytes reason);

function _registerTokenType(address validator) internal {
    if (validator != address(0)) {
        uint256 size;
        assembly { size := extcodesize(validator) }
        if (size > 0) {
            try ITransferValidatorSetTokenType(validator).setTokenTypeOfColle
ction(address(this), _tokenType()) {
                // (optional) emit success event
            } catch (bytes memory reason) {
                // Log failure instead of swallowing it
                emit TokenTypeRegistrationFailed(validator, address(this), re
ason);
                // (optional) if called during init and strict mode is desire
d:
                // revert("Token type registration failed");
            }
        }
    }
}
```

This preserves the current non-blocking behavior while giving operators and monitoring tools a clear signal when registration fails.

Resolution: The finding was fixed in the commit hash `f070187`. The empty catch block was replaced with event emitting.

F-2025-13272 - NatSpec Inconsistencies and Documentation Issues - Info

Description:

Multiple NatSpec documentation inconsistencies have been identified across all in-scope contracts. These inconsistencies range from missing parameter documentation to incorrect function descriptions and contract name mismatches. While these issues don't directly affect contract functionality, they significantly impact code maintainability, developer experience, and can lead to integration errors.

1. Contract Title Mismatch - PaniniNFTs

- **Location**: `src/PaniniNfts/PaniniNFTs.sol:19`
- **Issue**: NatSpec says `@title PhoenixNFTs` but contract name is `PaniniNFTs`
- **Impact**: Documentation confusion

2. Missing Parameter Documentation - PaniniNFTs.initialize()

- **Location**: `src/PaniniNfts/PaniniNFTs.sol:82-86`
- **Issue**: Missing `@param nftManager` documentation
- **Current NatSpec**:

```
/**
 * @notice Initializes the NFT contract with royalty and access control
 * @param initialOwner Address to be assigned as the initial contract owner
 * @param receiver Address to receive royalty fees
 * @param feeNumerator Royalty fee (basis points format, e.g., 500 = 5%)
 */
```

- **Missing**: `@param nftManager Address to be granted NFT manager role`

3. Wrong Function Description - PaniniNFTs.updateMintStatus()

- **Location**: `src/PaniniNfts/PaniniNFTs.sol:448-452`
- **Issue**: NatSpec says "Enables or disables burning functionality" but function controls minting
- **Current NatSpec**:

```
/**
 * @notice Enables or disables burning functionality.
 * @param _status True to enable burn, false to disable.
 * @dev Can only be called by onlyOwner.
```

```
 */
function updateMintStatus(bool _status) public onlyOwner {
```

- **Should be**: "Enables or disables minting functionality"

4. Incorrect Parameter Description - PaniniRoyaltyVault.initialize()

- **Location**: `src/PaniniRoyaltyVault/PaniniRoyaltyVault.sol:73`
- **Issue**: `@param _whitelistedAccount Vault managers` is incorrect
- **Actual Usage**: Parameter is used for whitelisted receiver, not vault manager
- **Should be**: `@param _whitelistedAccount Address to be whitelisted as receiver`

5. Missing Parameter Documentation - PaniniRoyaltyVault.initialize()

- **Location**: `src/PaniniRoyaltyVault/PaniniRoyaltyVault.sol:71-76`
- **Issue**: Missing `@param _pauser` documentation
- **Current NatSpec**:

```
/**
 * @notice Initializes the contract with initial configurations.
 * @param _whitelistedAccount Vault managers.
 * @param _uniswapRouter Address of the Uniswap V2 router.
 * @param _owner Address that will become the owner.
 */
```

- **Missing**: `@param _pauser Address to be granted pauser role`

6. Parameter Order Mismatch - PaniniRoyaltyVault.initialize()

- **Location**: `src/PaniniRoyaltyVault/PaniniRoyaltyVault.sol:71-82`
- **Issue**: NatSpec parameter order doesn't match function signature
- **NatSpec Order**: `_whitelistedAccount`, `_uniswapRouter`, `_owner`
- **Function Order**: `_owner`, `_pauser`, `_whitelistedAccount`, `_uniswapRouter`

7. Missing Parameter Documentation - PaniniRoyaltyVault.swapEthForToken()

- **Location**: `src/PaniniRoyaltyVault/PaniniRoyaltyVault.sol:196-202`
- **Issue**: Complete absence of parameter documentation
- **Current NatSpec**:

```
/**
 * @notice Swaps ETH for a whitelisted ERC20 token and sends it to the recipi
```

```
ent.
 */
```

- **Missing**: All `@param` documentation for `amountIn`, `outToken`, `amountOutMin`, `recipient`

8. Missing Function Documentation - PaniniRoyaltyVault.updateUniswapRouter()

- **Location**: `src/PaniniRoyaltyVault/PaniniRoyaltyVault.sol:268-273`
- **Issue**: No documentation whatsoever for critical function
- **Missing**: `@notice`, `@param`, `@dev` documentation

9. Missing Function Documentation - SmartValidator.setPaniniLock()

- **Location**: `src/PaniniNfts/smartlocks/SmartValidator.sol:64-67`
- **Issue**: Critical function with no documentation
- **Missing**: `@notice`, `@param`, `@dev`, access control documentation

10. Malformed NatSpec Comment - SmartValidator

- **Location**: `src/PaniniNfts/smartlocks/SmartValidator.sol:14`
- **Issue**: Uses `//` instead of proper NatSpec format
- **Current**: `// @notice Initializes the contract instead of constructor`
- **Should be**: `/// @notice Initializes the contract instead of constructor`

11. Contract Name Mismatch - CreatorTokenValidator

- **Location**: `src/PaniniNfts/limitbreak/CreatorTokenValidator.sol:11`
- **Issue**: NatSpec says `@title CreatorTokenBase` but contract is `CreatorTokenValidator`

12. Missing Function Documentation - PaniniNFTs.getRequestNonceStatus()

- **Location**: `src/PaniniNfts/PaniniNFTs.sol:485-489`
- **Issue**: Public function with no documentation
- **Missing**: `@notice`, `@param`, `@return` documentation

Assets:

- contracts/PaniniNfts/PaniniNFTs.sol [https://github.com/paniniamerica/SmartContracts]
- contracts/PaniniRoyaltyVault/PaniniRoyaltyVault.sol [https://github.com/paniniamerica/SmartContracts]
- contracts/PaniniNfts/smartlocks/SmartValidator.sol [https://github.com/paniniamerica/SmartContracts]
- contracts/PaniniNfts/limitbreak/CreatorTokenValidator.sol [https://github.com/paniniamerica/SmartContracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider fixing all NatSpecs in the code according to the suggested fixes.

Resolution: The finding was resolved in commit hash `f070187` after the suggested fixes were implemented.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken’s ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project’s performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/paniniamerica/SmartContracts
Initial commit	4b9b35def995b0de48288fe22ed85fd3a934f5c0
Remediation commit	f070187a1a8eb96f28f8e9c3b4b7825f233b6a1e
Final commit	68c618bf5a5f6a69301819973cb474320043830a
NFT Contract Proxy Address	https://etherscan.io/address/0x23ae7a05f598fc234ee9dbef04033080dea8ab19
NFT Contract Contract Implementation Address:	https://etherscan.io/address/0x290ae0178314705d95e504ea94ce052fb550b613
Royalty Vault Contract Proxy Address:	https://etherscan.io/address/0x2033b17894bd32af9297246827d43d9d67260af3
Royalty Vault Contract Implementation Address:	https://etherscan.io/address/0x71cf823dfeb46dc24ed16d91b8b5fcc7387a2c4e
Whitepaper	N/A
Requirements	PaniniNftAuditDoc.md, PaniniRoyaltyVaultAuditDoc.md, README.md
Technical Requirements	PaniniNftAuditDoc.md, PaniniRoyaltyVaultAuditDoc.md, README.md

Asset	Type
contracts/PaniniNfts/limitbreak/CreatorTokenValidator.sol [https://github.com/paniniamerica/SmartContracts]	Smart Contract
contracts/PaniniNfts/PaniniNFTs.sol [https://github.com/paniniamerica/SmartContracts]	Smart Contract
contracts/PaniniNfts/smartlocks/SmartValidator.sol [https://github.com/paniniamerica/SmartContracts]	Smart Contract
contracts/PaniniRoyaltyVault/PaniniRoyaltyVault.sol [https://github.com/paniniamerica/SmartContracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.